

IEM NEWTOWN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

ASSIGNMENT 5

Course: Python for Automation

Name	Roshni Ghosh Tagore
Enrollment No	12023002028175
Course	Python for Automation
Academic Year	2023-2027
Department	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus	IEM Newtown

TIER 2: ADVANCED USER INTERACTIONS

Assignment 5: The HTML Web Table Extractor

1. PROBLEM STATEMENT

In modern web test automation and web scraping workflows, dynamic web tables present complex data structures where content arrangement changes based on backend data. The requirement is to write an automated script using Selenium in Python to locate a specific data record within a multi-column dynamic web table by matching a row descriptor (e.g., matching the Last Name string "Smith") and extracting the corresponding dynamic value from another column (e.g., the "Due" column) within that targeted row.

2. OBJECTIVE

- Launch Google Chrome browser programmatically using Selenium WebDriver.
- Navigate to the dynamic practice page at <https://the-internet.herokuapp.com/tables> and maximize the browser window.
- Identify and extract all row elements from the target web table using relative XPath.
- Iterate through each row's cells (<td>) to find a record matching the Last Name "Smith".
- Retrieve and log the target value from the 4th column (Due Amount) corresponding to the located row.
- Gracefully terminate the browser session using `driver.quit()`.

3. TOOLS, SOFTWARE, AND CONCEPTS USED

Programming Language: Python 3.x

Automation Framework: Selenium WebDriver (`selenium.webdriver`)

Locators Strategy: XPath (By `.XPATH`) and Tag Name (By `.TAG_NAME`)

Browser & Driver: Google Chrome & ChromeDriver

Development Environment: Visual Studio Code with Python Virtual Environment (`.venv`)

Core Concepts: Dynamic DOM Traversal, Nested Web Elements Searching, Table Row/Column Parsing, and Conditional Logic.

4. SOURCE CODE OR IMPLEMENTATION STEPS

Implementation Steps:

1. Import the `webdriver` module and the `By` locator class from `selenium`.
2. Instantiate the Chrome WebDriver using `webdriver.Chrome()` and open the table URL.
3. Fetch all table body row elements using XPath: `//table[@id='table1']/tbody/tr`.
4. Loop through each row element and extract its child column cells using `row.find_elements(By.TAG_NAME, "td")`.
5. Check if the text in the first column (index 0 - Last Name) equals "Smith".
6. If matched, fetch the text from the fourth column (index 3 - Due) and display the output in the console.
7. Exit the loop and safely close the browser session using `driver.quit()`.

Source Code (assignment5.py):

```

from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

driver.get("https://the-internet.herokuapp.com/tables")
driver.maximize_window()

# Find all rows
rows = driver.find_elements(
    By.XPATH, "//table[@id='table1']/tbody/tr"
)

search_name = "Smith"

# Go through each row
for row in rows:

    # Find all columns in the current row
    columns = row.find_elements(By.TAG_NAME, "td")

    # First column = Last Name
    last_name = columns[0].text

    if last_name == search_name:

        # Fourth column = Due
        value = columns[3].text

        print("Name:", last_name)
        print("Value:", value)

        break

driver.quit()

```

5. OUTPUT WITH CLEARLY LABELLED SCREENSHOTS

```
assignment5.py > ...
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3
4  driver = webdriver.Chrome()
5
6  driver.get("https://the-internet.herokuapp.com/tables")
7  driver.maximize_window()
8
9  # Find all rows
10 rows = driver.find_elements(
11     By.XPATH, "//table[@id='table1']/tbody/tr"
12 )
13
14 search_name = "Smith"
15
16 # Go through each row
17 for row in rows:
18
19     # Find all columns in the current row
20     columns = row.find_elements(By.TAG_NAME, "td")
21
22     # First column = Last Name
23     last_name = columns[0].text
24
25     if last_name == search_name:
26
27         # Fourth column = Due
28         value = columns[3].text
29
30         print("Name:", last_name)
31         print("Value:", value)
32
33         break
34
35 driver.quit()
```

Figure 1: Visual Studio Code Editor displaying the complete Python automation script (assignment5.py)

```
(.env) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots> python -u "c:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots\assignment5.py"
Name: Smith
Value: $50.00
(.env) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots> █
```

Figure 2: PowerShell Terminal execution log showing successful extraction (Name: Smith | Value: \$50.00)

6. BRIEF RESULT, OBSERVATION, AND CONCLUSION

Brief Result:

The automation script executed without errors, successfully navigated to the practice web page, extracted all dynamic rows from table1, located the row matching "Smith", and extracted the corresponding Due value of "\$50.00".

Observation:

- Using relative XPath `//table[@id='table1']/tbody/tr` successfully handles dynamic table sizes and isolates the targeted table from other tables on the page.
- Chaining element finding (calling `row.find_elements(By.TAG_NAME, "td")` on a row instance) efficiently parses child nodes relative to the parent row without needing complex global XPaths for individual cells.

Conclusion:

The lab experiment successfully fulfilled the task requirements of dynamic web table extraction using Python and Selenium WebDriver. The approach provides a robust pattern for automated data verification and extraction in complex web applications.